Campus Café Ordering System

Code Explanation — Java OOP Laboratory Exercise

1. Overview

The program processes café orders using seven classes. **MenuItem** is an abstract parent of **Beverage**, **Pastry** and **Meal**. **Customer** stores the buyer's details, **Order** links a customer to up to three items, and **CafeApp** runs the demonstration. No ArrayList or other collection is used; **Order** keeps its items in three separate fields.

2. MenuItem (abstract class)

- **name** and **basePrice** are *protected* so subclasses can use them; **itemCode** is *private* (data hiding).
- **computePrice()** is abstract: every subclass must define its own pricing. This is **abstraction**, and MenuItem cannot be instantiated.
- **getDescription()** returns the name and item code, and subclasses extend it.
- **setBasePrice()** uses if-else to reject negative values and prints an error message.
- A default constructor ("Unnamed Item", 0.0, "N/A") and a parameterized constructor are provided.

3. Beverage, Pastry and Meal (subclasses)

Each subclass calls **super(...)** in its constructors, then overrides **computePrice()** and **getDescription()** with @**Override**. This is method overriding and gives **polymorphism**: Order can call computePrice() on any item without knowing its type.

Class	Pricing rule	Conditional used	Setter validation
Beverage	S = base, M = base + 15, L = base + 25	if / else-if / else chain on size	setSize accepts only S, M, L; otherwise keeps the old value
Pastry	base, plus 10 if warmed	single if	none needed (boolean)
Meal	base + 20 if upsized + 12 per extra side	if for the upsize fee	setExtraSides rejects negative numbers

The default constructors call **super()** and set safe starting values (for example, a Beverage starts as size "S").

4. Customer

- **Constructor overloading:** Customer(), Customer(name) and Customer(name, student). They chain together with **this(...)**, so the setup code is written once.
- **Method overloading:** addPoints(int) adds points directly; addPoints(double) converts pesos to points (1 per 50 pesos, using an (int) cast to drop the fraction).
- Setters validate input, and loyaltyPoints can never become negative.

5. Order

- **Object interaction:** Order stores a Customer and three MenuItem references. Because the fields have the parent type, any Beverage, Pastry or Meal fits.
- **addItem()** uses an if-else-if chain to fill the first empty slot. If all three are taken, it prints "Order is full" and returns false.
- **computeSubtotal()** adds computePrice() for each non-null item (polymorphic calls).
- **computeTotal()** subtracts the 10% student discount. **computeTotal(double voucher)** is the overloaded version that also subtracts a voucher and never returns less than 0.
- **printReceipt()** prints each item, the subtotal, discount, voucher and total, then calls customer.addPoints(total). The Customer's data changes because of the Order.
- A validated **setVoucher()** stores the voucher so printReceipt() can use it without extra parameters.

6. CafeApp (main)

- Creates two Customers with different constructors: Ana (name, student) and Ben (name only).
- Creates items with parameterized constructors (Iced Latte, Ensaymada, Chicken Adobo) and with default constructors filled by setters (Hot Chocolate, Beef Tapa, Cookie).
- Tries to add a fourth item to Order 1001 to show the limit check.
- Calls setBasePrice(-5) to show a setter rejecting invalid data, then setBasePrice(80).
- Prints both receipts.

7. Sample Computations

Order	Items	Subtotal	Adjustment	Total	Points
#1001 Ana (student)	Latte L 120 + Ensaymada warmed 55 + Adobo upsized, 1 side 142	317.00	-31.70 (10%)	285.30	5
#1002 Ben	Hot Chocolate M 95 + Beef Tapa 130	225.00	-20.00 voucher	205.00	4

Points are the total divided by 50, rounded down: 285.30 / 50 = 5, and 205.00 / 50 = 4.

8. Concept Map

Concept	Where it appears
if statement	Pastry.computePrice(); Meal.computePrice()
if-else	MenuItem.setBasePrice(); Beverage.computePrice(); Order.addItem(); Order.computeTotal()
Classes and objects	Seven classes; objects created in CafeApp.main()
Object interaction	Order holds a Customer and MenuItems; printReceipt() updates the Customer's points
Default / parameterized constructors	All MenuItem subclasses, Customer, Order
Constructor overloading	Three Customer constructors

Concept	Where it appears
Access modifiers and data hiding	Private fields, protected name and basePrice, validated setters, no public fields
Method overloading	Customer.addPoints(int/double); Order.computeTotal() / computeTotal(double)
Method overriding	computePrice() and getDescription() in Beverage, Pastry, Meal
Abstraction	abstract class MenuItem with abstract computePrice()
Naming conventions	PascalCase classes, camelCase members, UPPER_SNAKE_CASE constants (MAX_ITEMS, STUDENT_DISCOUNT_RATE)

9. Explanation of if and if-else

if runs a block only when its condition is true. If the condition is false, Java skips the block and continues. In **Pastry.computePrice()**:

```
double price = basePrice;
if (warmed) {
    price += 10;   // only runs when warmed is true
}
return price;
```

An unwarmed pastry simply keeps its base price.

if-else chooses between two paths, and exactly one of them runs. In **MenuItem.setBasePrice()**:

```
if (basePrice < 0) {
    System.out.println("Error: base price cannot be negative.");
} else {
    this.basePrice = basePrice;
}
```

A negative price triggers the error and is never stored. Any other price is saved.

if-else-if chain handles three or more options. Java checks from top to bottom and runs the first true branch; the final else catches everything else. In **Beverage.computePrice()**:

```
if (size.equals("L")) {
    return basePrice + 25;
} else if (size.equals("M")) {
    return basePrice + 15;
} else {
    return basePrice;   // S (or anything else)
}
```

When to use which

- Use **if** alone for an optional extra, such as a fee.
- Use **if-else** for a two-way choice, such as valid or invalid.
- Use **if-else-if** for several exclusive cases, such as sizes or empty slots.

Other places in the project

- **Order.addItem()** uses an if-else-if chain to fill slot 1, then 2, then 3, otherwise report "Order is full".
- **Order.computeTotal(voucher)** uses if-else so the total never goes below 0.
- **Meal.computePrice()** uses if for the upsize fee.